The Shaders Folder

How The Silent Horn's GPU effects are built, connected and driven

`src/shaders/ · 6 files · three.js ShaderMaterial + GLSL`

Six small files hold the game's custom GPU effects. Each is a three.js `ShaderMaterial` with its GLSL vertex and fragment shaders written inline as JavaScript strings. A level imports a shader, puts it on a mesh, and updates a few numbers on it every frame.

> **Three things to remember.** (1) Every shader file has the **same structure**. (2) Every fragment shader follows the **same six-step recipe**. (3) Levels drive every shader the **same way**: `uTime` to animate it, plus one 0–1 game-state value to link it to gameplay.

1. At a glance

File	What you see on screen	Used in
`noise.js`	Nothing on its own: the shared noise library the other shaders build on	water, waterfall, rain, light shafts, plus the sky and the river
`jungleSpeedWarpShader.js`	Speed streaks along the edges of the trail	Level 1
`rain.js`	Rain streaks across the camera lens	Level 2
`waterfall.js`	A falling curtain of water with foam	Level 2 (the gorge), Level 3
`water.js`	A rippling pool surface with sun glints and foam	Level 2 (the gorge), Level 3
`lightshaft.js`	God rays of sunlight with drifting dust	Level 3

2. How a shader runs

A shader is a small program that runs on the graphics card. The game code on the CPU only sets it up and feeds it numbers; the GPU runs it for every vertex and every pixel, in parallel.

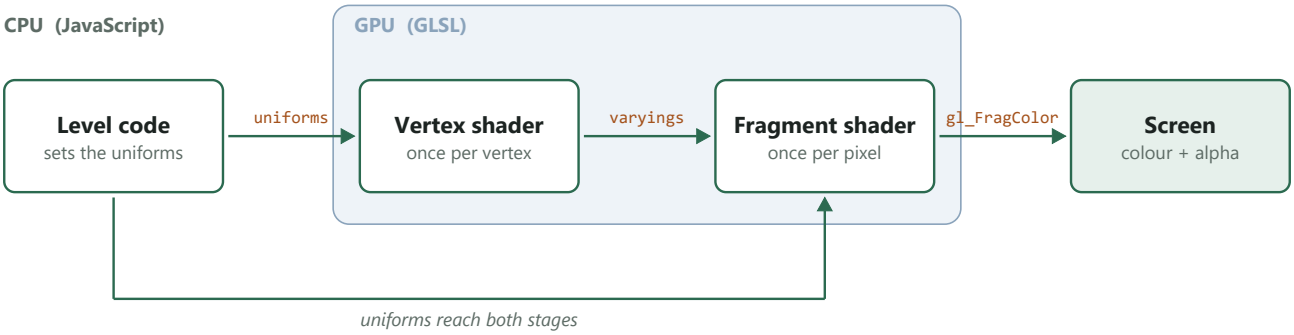


Figure 1. Data flow for every shader in the folder.

- **Uniforms** are values the CPU sends in: the same for every vertex and pixel in a frame (`uTime`, `uSpeed`, colours).
- **The vertex shader** runs once per vertex. It moves the vertex to its place on screen (object → world → camera → clip space) and hands data on.
- **Varyings** carry that data to the fragment shader, blended smoothly across each triangle (`vUv`, `vWorld`).
- **The fragment shader** runs once per pixel and decides its colour and transparency. Almost all the work in this folder happens here.

3. The reusable pattern

Every file in the folder is built from the same parts, in the same order. Once you can read one, you can read them all, and adding a new effect means filling in the same template.

3.1 Anatomy of a shader file

This skeleton is the shape shared by `water.js`, `waterfall.js`, `rain.js`, `lightshaft.js` and the speed warp. The numbered parts are explained in the table below.

```
import * as THREE from 'three';
import { NOISE_GLSL } from './noise.js';                 // [1] shared GLSL

/**
 * What the effect looks like, and how a level drives it:   [2] usage contract
 *   mat.uniforms.uTime.value = seconds;
 *   mat.uniforms.uTint.value = 0..1;
 */
const vertex = /* glsl */ `                              // [3] vertex stage
  varying vec2 vUv;
  void main() {
    vUv = uv;                                            // hand data on to the fragment stage
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
  }
`;

const fragment = NOISE_GLSL + /* glsl */ `               // [4] fragment stage
  uniform float uTime;                                   // animation
  uniform float uTint;                                   // game state, 0..1
  uniform vec3 uColor;                                   // look
  varying vec2 vUv;
  void main() {
    // ...the six-step recipe from section 3.3...
    gl_FragColor = vec4(colour, alpha);
  }
`;

export function createExampleMaterial({ color = 0x52a0a8 } = {}) {   // [5] factory
  return new THREE.ShaderMaterial({
    vertexShader: vertex,
    fragmentShader: fragment,
    transparent: true,                                   // [6] render state
    depthWrite: false,
    uniforms: {                                          // [7] inputs, with defaults
      uTime:  { value: 0 },
      uTint:  { value: 0 },
      uColor: { value: new THREE.Color(color) },
    },
  });
}
```

Figure 2. The template every shader file follows (simplified from waterfall.js).

	Part	What it is for
[1]	Shared GLSL	GLSL has no `import`, so shared functions live in a string (`NOISE_GLSL`) that is pasted in front of the shader.
[2]	Usage contract	The comment at the top says what the effect looks like and which uniforms a level must set. Every file opens with one.
[3]	Vertex stage	Places each vertex on screen and passes the UV or world position on. Short in every file.
[4]	Fragment stage	Builds each pixel's colour. The `/* glsl */` tag makes the editor highlight the string as GLSL.
[5]	Factory function	The only thing a level imports. Options with defaults let one shader serve several places (e.g. two different pools).

	Part	What it is for
[6]	Render state	Every material is `transparent` with `depthWrite: false`, so see-through effects layer without cutting holes in each other. Light effects add `AdditiveBlending`.
[7]	Uniforms	The shader's inputs, each with a starting value. After this, the level only ever changes `.value`.

3.2 The level side: set up once, update every frame

A level uses a shader in two places: once in its setup, and then in its per-frame `update`. This is the real code for the Level 3 pool, lightly trimmed:

```
// setup, once                                          (ShrineArena._buildWater)
this.poolMat = createWaterMaterial({ sunDir: SHRINE_DAY.sunDir, foamAt });
const pool = new THREE.Mesh(new THREE.CircleGeometry(POOL.r, 64), this.poolMat);
this.root.add(pool);

// every frame                                          (ShrineArena.update)
this.flowTime += dt * (1 + this.dusk * 0.9);          // flows faster in the last phase
this.poolMat.uniforms.uTime.value = this.flowTime;    // animation

// when the light changes                               (ShrineArena.setDusk)
this.poolMat.uniforms.uTint.value = k * 0.75;         // game state: the pool runs red
```

Figure 3. Create, attach, then drive through uniforms.

The shader never reads game objects. The level pushes numbers into it. That is what makes the shaders reusable: the same `createWaterMaterial()` is the Level 3 pool and the pool at the bottom of the Level 2 gorge, just with different colours and flow options.

One file goes a step further and exports the per-frame update as well: `updateJungleSpeedWarp(material, dt, speed)` advances `uTime` and clamps the speed to 0–1, so Level 1's update is a single line.

3.3 Inside the fragment shader: a six-step recipe

Read any fragment shader in the folder from top to bottom and you will find these steps, in this order. Only step 3, the pattern, really changes from file to file. Examples are from `waterfall.js`.

Step	What it does	Example (waterfall.js)
1. Coordinates	Where on the surface is this pixel?	`vUv` (texture position) or `vWorld.xz` (world position)
2. Animate	Add `uTime` so the pattern moves	`vUv.y * 2.2 + uTime * 2.4`
3. Pattern	`noise` / `fbm` for natural shapes, `fract` for repeats	`fbm(vec2(vUv.x * 14.0, ...))`
4. Shape	`smoothstep` turns a soft pattern into foam, streaks or soft edges	`smoothstep(0.45, 0.8, streak)`
5. Colour	`mix` between colours; the game-state uniform mixes in another	`mix(uWater, uBlood, uTint)`
6. Output	Colour + transparency, then three.js tone mapping and fog	`gl_FragColor = vec4(c, a);`

3.4 Three kinds of uniform

Every uniform in the folder falls into one of three groups.

Kind	Purpose	Set by	Examples
Animation	Makes the effect move	The level, every frame	`uTime`
Game state	Links the effect to gameplay. Always 0–1	The level, when the game changes	`uSpeed`, `uIntensity`, `uTint`, `uStrength`
Look	Colours and placement	The factory's options (a level can still change them)	`uColor`, `uDeep`, `uSunDir`, `uFoamAt`, `uSeed`

Game-state uniforms are always normalised to **0–1**. The level does the maths (for example current speed ÷ top speed), so the shader only has to know "none" through "full".

3.5 Naming conventions

- `u` prefix = uniform (`uTime`, `uTint`); `v` prefix = varying (`vUv`, `vWorld`).
- `createXMaterial()` = the factory a level calls. (`createLightShaft()` returns a whole mesh instead, because it also builds the cone.)
- `/* glsl */` in front of each template string = highlight this string as GLSL.
- One effect per file, named after what it draws.

3.6 How each file fits the pattern

Same template, different choices. This is the whole folder on one table:

File	Coordinates	Pattern	Game state	Blending	What's special
noise.js	n/a	`hash`, `noise`, `fbm`	n/a	n/a	A library: just a GLSL string, no material
speed warp	`vUv`	`fract` lines	`uSpeed`	additive	Also exports its per-frame update
rain	`vUv` (screen)	3 noise layers	`uIntensity`	additive	Full-screen overlay; draws nothing when dry
waterfall	`vUv`	2 `fbm` layers	`uTint`	normal	The only one that moves vertices
water	`vWorld.xz`	`fbm` height field	`uTint`	normal	Builds normals from noise; uses scene fog
light shaft	`vUv` + view angle	noise dust	`uStrength`	additive	Factory returns a full cone mesh

3.7 Adding a new shader in five steps

1. Copy the skeleton (Figure 2) into a new file in `src/shaders/`.
2. Write the fragment shader with the six-step recipe. Put `NOISE_GLSL +` in front if it needs noise.
3. List its uniforms, with defaults, in the factory. Give the game-state one a 0–1 range.
4. In the level's setup: create the material, put it on a mesh, add the mesh to the scene.
5. In the level's `update`: advance `uTime` and set the game-state uniform.

4. How the files connect

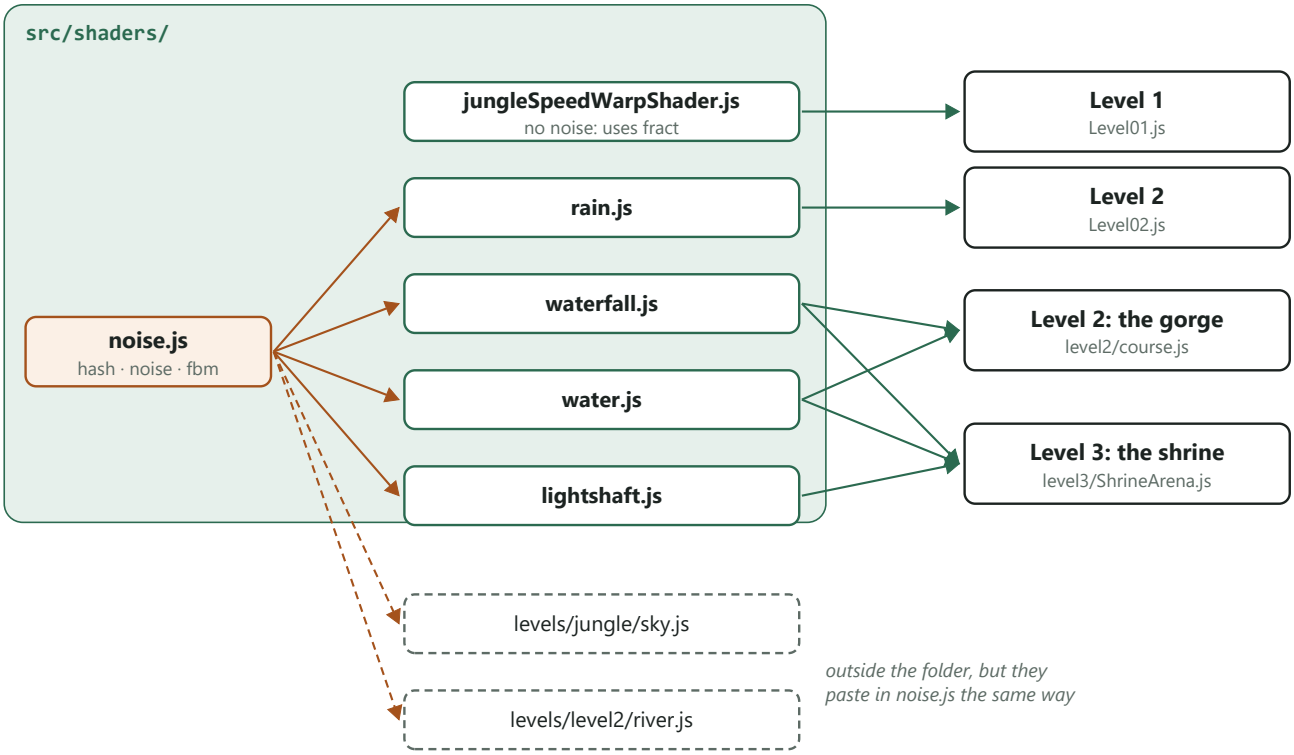


Figure 4. Orange arrows: shaders that paste in noise.js. Green arrows: which level uses each shader.

- `noise.js` **is the hub.** It exports one GLSL string, `NOISE_GLSL`, and the other shaders paste it in front of their own code. The jungle sky and the Level 2 river (outside the folder) reuse it the same way.
- **The speed warp is the only shader without noise.** Its streaks are built from repeating lines (`fract`) instead.
- **Water and waterfall are a pair.** In Level 3 the waterfall lands in the pool, and the pool's foam ring (`uFoamAt`) is placed where the water hits. The same pair appears at the end of Level 2, where the car drives over the falls.

Game state → uniform

Shader	Uniform	Driven by
Speed warp	`uSpeed` (0–1)	Kai's current speed: faster running gives more, quicker streaks
Rain	`uIntensity` (0–1)	How wet Level 2 has become: the rain fades in partway through
Water	`uTint` (0–1)	Level 3's final phase, when the pool turns blood red
Waterfall	`uTint`, and how fast `uTime` grows	The final phase: the fall runs red and flows faster
Light shaft	`uStrength` (0–1)	Time of day: the beams fade out at dusk

5. What each file does

`noise.js` the shared toolkit

- `hash(p)` returns a pseudo-random number for a grid cell.
- `noise(p)` gives smooth *value noise*: random values at the grid corners, blended smoothly in between.
- `fbm(p)` (*fractal Brownian motion*) stacks 5 layers of noise, each with twice the detail and half the strength. This gives the natural, cloudy look used for water, clouds, foam and dust.

`jungleSpeedWarpShader.js` speed streaks · Level 1

- Two glowing ribbons lie flat along the edges of the path.
- `fract()` cuts each ribbon into repeating cells with a thin line in each; the lines scroll with time to make streaks.
- As `uSpeed` rises the streaks get faster, thinner, more numerous and brighter. At walking speed they disappear.
- Additive blending means it only brightens the trail and never hides it.

`rain.js` rain on the lens · Level 2

- A full-screen quad, drawn in its own overlay scene with an orthographic camera, so it sits in front of everything.
- Three layers of noise, squashed thin and tall, scroll downwards at different speeds: fine streaks, bigger drops, mist.
- A larger, slow noise varies the density across the screen, so the rain looks uneven, like real rain.
- While `uIntensity` is 0 it discards every pixel, so it costs nothing before the rain starts.

`waterfall.js` the water curtain · Levels 2 and 3

- **The only shader that moves vertices:** the top of the sheet is pushed outward (`pow(uv.y, 2.0)`), so the fall curves away from the cliff instead of looking like a flat card.
- Two `fbm` streak layers scroll down at different speeds, and white foam appears where they peak.
- White churn at the base where it hits the pool, and soft fades at the sides and top.
- `uTint` mixes in red for Level 3's final phase.

`water.js` the pool surface · Levels 2 and 3

- **No normal-map texture.** The ripples come from a noise *height field*, and the surface normal is computed from its slope (finite differences).
- **Fresnel:** looking straight down you see the deep water colour; at shallow angles you see the sky reflected.
- **Sun glint:** a Phong specular highlight off the ripples.
- **Foam ring** around the spot where the waterfall lands.
- Uses three.js's own fog, tone-mapping and colour-space chunks (`#include <fog_fragment>`), so it matches the rest of the scene.

`lightshaft.js` god rays · Level 3

- A cone (an open cylinder) runs from the sky down to the ground.
- Brightest along its centre and soft at the rim, using an inverse-fresnel term (`dot(normal, view)`).
- Fades in at the top and out at the ground. Noise makes dust drift down the beam, and a slow `sin` makes it pulse gently.
- `uSeed` gives each beam its own timing, so they don't pulse together.

6. Techniques used throughout

Technique	What it gives	Where
Noise and fbm	Natural, non-repeating shapes	Water, foam, rain, dust, clouds
`smoothstep` **fades**	Soft edges instead of hard lines	Every shader
Scrolling with `uTime`	Movement	Every shader
Additive blending	Light that only brightens what's behind it	Rain, speed streaks, god rays
Fresnel	Looks that change with the viewing angle	Water reflections, light-shaft rims
`transparent` + `depthWrite: false`	See-through effects that layer cleanly	Every material

> **In one sentence:** `noise.js` provides shared randomness, and the other five shaders use it (or simple maths) to draw water, rain, light and speed. Each one is a three.js material built from the same template, which a level animates with `uTime` and links to gameplay through one 0–1 game-state uniform.